

layer

Serving
engines

- { Software layer that executes the model

→ choosing the right one is crucial
decision for performance, cost, scalability

Categories of Serving Engines

Local
Engines

- ollama, llama.cpp

→ best use: For local experiments,
CPU/mac workflows, edge, rapid testing

* Limitations: Not designed For high concurrency (ability to handle multiple task at the same time)
Production, multi-tenant inference, 1 GPU serves multiple user
advanced GPU scheduling

Production
OS Serving
engines

: VLLM, SG Lang, HF TGI, LM Deploy

→ Designed for high throughput
Continuous batching, prefix caching
Multi-GPU Setup

Vendor
optimized
engines

: TensorRT-LLM

→ Built for maximum performance, Latency
Sensitive workloads GPUs

high complexity:

how hard to build, deploy,
fix when it Break

→ we don't just run models

weights → we need to make

heavy Reliance on hardware specific
Optimization → math, factors for each GPU

then in specific format → Building this takes time

From scratch

- also when you got an error in vendor you got Cuda, C++ error
which is hard to trace